

Helix: 通过 Max-Flow 通过异构 GPU 和网络 为大型语言模型提供服务

梅以轩

yixuanm@andrew.cmu.edu
卡内基梅隆大学 美国宾夕
法尼亚州匹兹堡

庄永浩

yzhuang2@andrew.cmu.edu
卡内基梅隆大学 美国宾夕
法尼亚州匹兹堡

苗旭鹏

xupeng@cmu.edu 卡
内基梅隆大学 美国宾夕法
尼亚州匹兹堡

杨俊成

juncheny@cs.cmu.edu 卡
内基梅隆大学 美国宾夕法
尼亚州匹兹堡

贾志豪 zhihao@

cmu.edu 卡内基梅隆
大学 美国宾夕法尼亚
州匹兹堡

拉什米·维纳亚克

rvinayak@cs.cmu.edu 卡
内基梅隆大学 美国宾夕法
尼亚州匹兹堡

抽象的

本文介绍了 Helix，这是一个在异构 GPU 集群中提供高吞吐量、低延迟大型语言模型 (LLM) 的分布式系统。Helix 背后的关键思想是将异构 GPU 和网络连接上的 LLM 推理计算公式化为有向加权图上的最大流问题，其节点代表 GPU 实例，边通过其能力捕获 GPU 和网络异构性。然后，Helix 使用混合整数线性规划 (MILP) 算法来发现高度优化的策略，为异构 GPU 上的大语言模型服务。这种方法允许 Helix 联合优化模型放置和请求调度，这是异构 LLM 服务中两个高度纠缠的任务。我们对 24 到 42 个 GPU 节点的多个异构集群的评估表明，与现有方法相比，Helix 将服务吞吐量提高了 3.3×，并将提示和解码延迟分别减少了 66% 和 24%。Helix 可从 <https://github.com/Thesys-lab/Helix-ASPLOS25> 获取。

CCS 概念：

- 计算机系统组织 → 云计算；
- 计算方法 → 人工智能；并行计算方法。

关键词：大语言模型服务、机器学习系统、分布式系统、云计算

ACM参考格式：

梅宜轩、庄永浩、缪旭鹏、杨俊成、贾志浩和 Rashmi Vinayak。2025. Helix: 通过 Max-Flow 在异构 GPU 和网络上为大型语言模型提供服务。在第 30 届 ACM 国际编程语言和操作系统架构支持会议的会议记录中，



This work is licensed under a Creative Commons Attribution International 4.0 License.

ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0698-1/25/03

<https://doi.org/10.1145/3669940.3707215>

表 1. 在现有同类服务系统中为 LLM 提供服务所需的最低 GPU 数量。我们使用一半的 GPU 内存来存储模型参数，另一半用于键值缓存。

LLMs	Num. of Parameters	Num. of L4s	Num. of A100s	Num. of H100s
LLaMA-2 [56]	70 billion	12	7	4
GPT-3 [1]	175 billion	30	18	9
Grok-1 [59]	314 billion	53	32	16
LLaMA-3 [10]	405 billion	68	41	21

表 2. Google Compute Engine 上 6 个区域中不同 GPU 实例的可用性 [11]。

Region	GPU Type					
	H100	A100 80GB	A100 40GB	L4	T4	V100
us-central-1	✓	✓	✓	✓	✓	✓
us-east-4	✓	✓	×	✓	✓	×
us-east-1	×	×	✓	✓	✓	✓
eu-west-3	✓	×	×	✓	✓	×
asia-ne-1	✓	×	✓	✓	✓	×
asia-ne-3	×	×	✓	✓	✓	×

第 1 卷 (ASPLOS '25), 2025 年 3 月 30 日至 4 月 3 日, 荷兰鹿特丹。ACM, 美国纽约州纽约市, 17 页。 <https://doi.org/10.1145/3669940.3707215>

1 简介

GPT-4 [1] 和 LLaMA-3 [28] 等生成式大语言模型 (LLM) 已经展示了在一系列应用领域创建自然语言文本的卓越能力，包括聊天机器人 [38]、编码助手 [26, 44] 和任务自动化 [15]。然而，现代大语言模型越大的模型尺寸和高计算要求使得在现代云平台上廉价而高效地为其提供服务变得具有挑战性。特别是，当今大多数 LLM 服务系统（例如 Orca [61] 和 vLLM [22]）都以同构 GPU 集群 [29] 为目标，其中所有 GPU 都是同一类型，并且具有相同的内存容量和计算资源。由于型号增加

表 3. 当今数据中心中部署的 GPU 的属性。我们报告 H100 和 A100 的 SXM 版本。数据收集自 NVIDIA 数据表 [32-35]。

GPU	FP16 (TFLOPs)	Memory (GB)	Bandwidth (GB/s)	Power (W)	Price (USD)
H100 [33]	1979	80	3350	700	25k ~40k
A100 [32]	312	40	1555	400	10k ~15k
L4 [34]	242	24	300	72	~3k
T4 [35]	65	16	300	70	~1k

由于大小不同，使用同构 GPU 为 LLM 提供服务需要越来越多的 GPU，如表 1 所示。此外，为行业中使用的最先进的 LLM 提供服务需要更多的资源。最近的研究表明，在单个云区域内分配如此规模的 GPU 变得越来越困难 [50, 60]。

由于 GPU 架构设计的进步以及随着时间推移的增量部署，现代云平台越来越多地由多种 GPU 类型组成。表 2 说明了 Google 计算引擎 [11] 中的异构 GPU 部署，其中数据中心配备了各种 NVIDIA GPU，包括 H100、A100、V100、L4 和 T4。这些异构 GPU 实例分布在世界各地的数据中心，与单个 GPU 类型相比，它们共同提供更大的内存容量和更多的计算资源，从而为 LLM 服务提供更易于访问和扩展的方法。如表 3 所示，八个 NVIDIA L4 GPU 可以提供与单个 NVIDIA H100 GPU 相当的 FP16 计算性能，同时提供更大的内存容量、更低的功耗和更具成本效益的价格点。此外，这些 GPU 的可用性在不同地区存在很大差异。我们根据经验发现，在 Google Cloud Engine 上，某些区域比其他区域更容易获得更高的配额和保护 GPU 实例，并且不同 GPU 的可用性差异很大（参见表 4）。

使用异构 GPU 的地理分布式 LLM 可以聚合来自多个区域的可用 GPU。这种方法不仅提高了资源利用率，还通过战略性地利用不同地理位置的最具成本效益的 GPU 实例，最大限度地降低了 LLM 服务成本。同样，还有一种趋势是使用志愿消费 GPU 来解决 GPU 稀缺问题 [9, 46, 62]。然而，与同构 GPU 实例相比，在地理分布式异构实例上部署 LLM 需要适应各种 GPU 设备和网络条件。

之前的工作已经介绍了几种用于在异构设备或地理分布式环境上运行机器学习计算的系统。然而，之前的尝试要么专注于长时间运行的训练工作负载 [16, 27, 39, 45, 64]，这些工作负载无法适应具有实时推理请求的 LLM 服务场景，要么专注于去中心化

表 4. 我们评估 Helix 期间美国东部和亚洲东南部的 GPU 配额和部署限制。我们测量两个不同时间段的部署限制。

Region	Quota			Max Deployed		
	A100 40GB	L4	T4	A100 40GB	L4	T4
us-east	8	8	16	0	0	16
asia-southeast	8	24	32	4	12	>20

服务于志愿者计算 [5, 6]，缺乏有效使用集群中的 GPU 和网络资源所需的全局协调。

为了通过异构 GPU 和网络有效地为 LLM 提供服务，我们提出了 Helix，一种用于高吞吐量、低延迟 LLM 服务的分布式系统。Helix 的关键思想是将在异构 GPU 和网络服务的 LLM 的执行制定为在不同 GPU 计算能力、内存容量以及复杂的 GPU 间连接的约束下的数据流问题。Helix 利用混合整数线性规划来确定这些约束下的最佳模型放置。为了适应异构性，Helix 引入了每个请求管道，其中每个请求都有自己独立的管道进行调度。基于流的制定和按请求管道的结合使 Helix 能够在异构和地理分布式 GPU 集群中实现高 GPU 利用率。我们将在第二部分讨论挑战和 Helix 的解决方案。3.

我们在 vLLM [22] 之上实现了 Helix，并在 24 到 42 个节点、最多 7 种不同节点类型的三个异构集群上对其进行了评估。我们评估的模型包括 LLaMA-1 30B 和 LLaMA-2 70B。与异构感知基线相比，Helix 将服务吞吐量提高了高达 3.3 $\times$ ，同时将平均提示和解码延迟降低了高达 66% 和 24%。

总之，我们的贡献是：

- 一种服务于异构和地理分布式 GPU 集群的 LLM 系统。
- 用于 LLM 服务的最大流量公式和用于优化模型放置的基于 MILP 的算法。
- 灵活的基于流的每个请求管道可最大限度地提高 GPU 利用率。
- 我们的技术的实施以及对各种 LLM 基准的评估。

2 背景

2.1 LLM架构和服务

当今大多数大语言模型使用仅解码器的 Transformer 架构 [7, 42]，该架构首先将自然语言查询转换为标记序列。然后，模型将每个标记转换为隐藏状态向量，其大小称为模型的隐藏大小。Transformer 模型由输入和输出嵌入以及一系列相同的 Transformer 层组成，每个层都包含一个自注意力和一个前馈块。自注意力块

计算每对标记之间的“亲和力”，并根据上下文相关性得分更新每个标记的隐藏状态。前馈块通过非线性函数独立修改每个令牌的隐藏状态。

给定输入序列，Transformer 模型计算下一个标记的概率分布，并从该概率分布中采样。因此，该模型应用自回归范例来生成整个输出序列：给定输入提示，模型运行多次迭代。在第一次迭代（称为提示阶段）时，模型处理所有提示标记并生成第一个输出标记。在随后的迭代（称为解码阶段）中，模型结合提示和之前生成的标记来预测下一个输出标记。当模型产生特殊的句尾信号((eos))时，此迭代过程停止。由于生成输出是不可预测的，因此在序列完全生成之前，确切的迭代次数仍然不确定。

除了不可预测的执行迭代之外，LLM 服务的另一个特点是高内存需求。自注意力块需要所有先前令牌的隐藏状态作为输入。为了存储新生成的令牌的隐藏状态（称为 KV 缓存），内存需求随着生成过程不断增加。

为了应对这些挑战，Orca [61]提出了迭代级调度，它在每次迭代时更新一个批次，以避免当请求完成但同一批次中的其他人需要更多迭代时资源保留；vLLM [22]引入了 PagedAttention，为具有相同页面的 KV 缓存管理内存，并仅在请求用完其所有页面时才分配新页面；多查询[47]和组查询注意[3]修改了自注意机制以减少为每个令牌存储的KV缓存的大小。

2.2 分布式模型服务

开源 LLM 现在具有多达数千亿个参数，远远超过单个 GPU 的内存容量。因此，为大语言模型服务需要多个 GPU 并行运行。张量并行性 (TP) [49]在 GPU 之间划分每个算子的权重，通过 AllReduce/AllGather 操作收集每个设备上的部分结果。然而，TP 对网络状况非常敏感。对于每个 Transformer 层，它需要两次通信。因此，TP 在高延迟网络中具有显着的开销，并且仅在节点内的 GPU 之间使用。

相反，管道并行性 (PP) [17] 在 GPU 上分配不同的运算符（通常是多个层）以创建多个管道阶段。然后它将输入分成微批次，并通过管道运行它们。PP 仅在管道阶段的边界传输激活张量。因此，PP 对网络的敏感度要低得多。然而，完美划分模型和输入批次具有挑战性，这会导致管道气泡。结果，PP 遭受了

设备在管道出现气泡时闲置，需要仔细安排才能提高性能 [2]。

传统的数据中心设置通常假设同质集群：具有统一带宽的统一节点。因此，模型被均匀地划分为管道阶段并分配给每个设备。随着大语言模型的不断扩大，而新一代 GPU 仍然稀缺，跨异构计算设备部署这些模型已成为一项至关重要的任务，而之前的研究尚未充分解决这一挑战。

跨地理分布式、异构 GPU 集群部署 LLM 需要仔细考虑硬件功能和网络特性。跨设备简单地平等分配模型层无法最大限度地发挥更强大硬件的潜力。此外，区域内和区域间网络连接之间的显著带宽差异必须为模型放置和请求调度决策提供信息，特别是当基础设施跨越多个地理区域时。

之前没有任何工作关注在异构 GPU 集群上服务的 LLM。最相关的工作是 Petals [5]，它通过志愿者计算执行去中心化的 LLM 服务。用户贡献 GPU 来形成分散的群，新加入的机器贪婪地以最少的计算能力服务于管道阶段。对于请求调度，用户贪婪地选择对他们来说延迟最低的服务器。Petals 对于志愿计算是有效的，但是缺乏全局协调使得在集群先验已知的情况下无法充分利用 GPU 和网络。另一个相关工作是 SWARM [45]，它在异构集群中进行 DNN 训练。它将模型均匀地划分为管道阶段。当将请求路由到下一个管道阶段时，它会根据候选者的实时吞吐量选择副本。我们的评估表明，这种简单的启发式方法不足以在具有异构 GPU 和网络的地理分布式集群中实现良好的性能。

3 机遇与挑战

使用异构和地理分布式 GPU 为 LLM 服务提供了新的机会。如表 3 所示，多种商用 GPU (L4、T4) 可以与高端 GPU (H100、A100) 的计算能力相匹配，同时在内存容量、能源效率和成本方面具有优势。此外，表 4 和表 7 表明，GPU 可用性在不同区域之间存在显著差异，但区域间网络条件仍然适合 LLM 服务，从而激发了地理分布式方法。然而，利用异构和地理分布式 GPU 带来了几个关键挑战。

3.1 挑战一：模型放置

由于 LLM 的规模不断增加，在现代 GPU 上为它们提供服务需要使用张量 [49] 和管道 [17, 39]

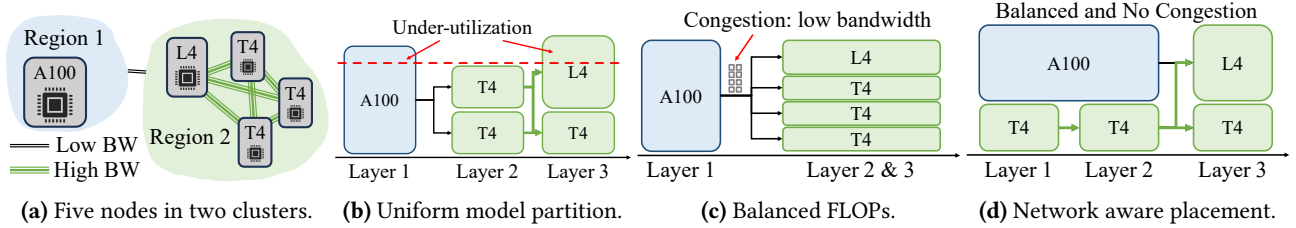


图 1. 次优模型放置和请求计划的示例。1a) 本例中的所有 GPU 和网络状况。计算能力顺序为: A100 > L4 > T4; 1b) 通过对模型进行统一分区来进行模型放置, 然后通过平衡计算能力来分配设备; 1c) 协同优化模型划分和设备布局, 使计算能力更加均衡; 1d) 以网络感知的方式共同优化模型分区、设备布局 and 请求调度。

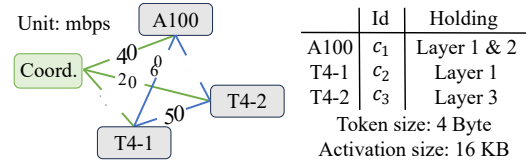
模型并行性将 LLM 划分为多个阶段并将这些阶段放置在不同的 GPU 上, 我们将这一任务称为模型放置。同质服务系统 (例如 Orca [61]) 将 LLM 划分为大小相等的阶段并将它们分配给 GPU。这种方法会导致高性能 GPU 的利用率不佳, 因为它适应了功能较弱的 GPU 的内存和计算限制。现有的异构感知服务系统 (例如 Petals [5]) 依靠不同的启发式方法将模型划分为多个阶段并将其分配给 GPU。现有的启发式方法并未同时考虑 GPU 和网络异构性。

Helix 的解决方案: Helix 利用 LLM 服务中令牌级调度的灵活性, 并将模型放置制定为有向加权图的最大流问题, 其节点代表 GPU 实例, 边通过其在最大流问题中的能力捕获 GPU 和网络异构性。然后, Helix 使用混合整数线性规划 (MILP) 算法来发现高度优化的模型放置策略, 该策略在很大程度上优于先前工作中使用的启发式方法 [5, 45]。利用 LLM 层的数据依赖性和同质性, Helix 用相对于计算节点和网络连接数量的线性变量和约束来表达 MILP 问题, 从而产生易于处理的问题大小。

3.2 挑战2: 请求调度

Helix 必须解决的第二个挑战是请求调度。为了满足 LLM 请求, Helix 需要选择 GPU 实例管道来计算 LLM 的所有层。现有系统通常采用一组固定管道, 并以循环方式将请求分配给这些管道。使用固定管道不够灵活, 无法适应异构计算和网络条件, 并且经常导致利用率不足。

Helix 的解决方案: Helix 引入了按请求管道, 其中每个请求都分配有自己的管道。因此, 潜在管道的总数等于集群图形表示中从源到接收器的路径数量, 这为 Helix 提供了足够的灵活性, 可以最大限度地利用 GPU 实例的全部容量以及它们之间的网络连接。



(a) 具有模型放置的 3 节点集群。协调器和计算节点之间的网络连接传输令牌 (4 字节), 而其他节点则传输中间激活 (16 KB)

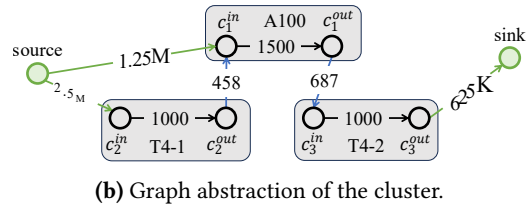


图 2. 具有给定模型放置的 3 节点集群的图形抽象。图 2 b 中边缘上的数字代表其容量, 即每秒可以通过边缘的令牌数量。源和宿之间的最大流量等于集群的最大服务吞吐量。

4 Helix 中的优化公式

本节首先提供 LLM 服务系统的数学抽象。基于这个公式, 我们将异构 LLM 建模为最大流问题。最后, 我们应用混合整数线性规划 (MILP) 来搜索具有最高最大流量的模型放置策略。

4.1 LLM 服务的制定

服务 LLM 的集群通常包含一个协调器节点 h 和一组计算节点 C 。每个计算节点 $c_i \in C$ 都有计算能力和 GPU VRAM 大小。具有多个 GPU 的计算节点可以抽象为单个逻辑节点, 聚合 GPU 的组合计算能力和 GPU VRAM 资源。还给出了集群中节点之间网络连接的吞吐量和延迟。基于集群信息, LLM 服务需要找到模型层到计算节点的位置, 以最大化下面将定义的服务性能指标。模型放置函数 $\Psi: C \mapsto \mathcal{P}(\mathcal{M})$ 采用

作为输入计算节点并返回模型 M 的（通常是连续的）子集。在这里，我们假设跨 GPU 的节点间并行化采用管道并行化，而跨 GPU 的节点内并行化采用张量并行化，因为张量并行化需要广泛的通信，并且可能受到网络的限制。评估模型放置的一种广泛使用的指标 [5, 45] 是具有最低计算能力 $\min_i \sum_j \text{容量}(j) \cdot \mathbf{1}_{i \in \Psi(j)}$ 的管道阶段的性能，其中容量(j) 是 j^{th} 节点的计算能力。正如我们将在下面展示的，仅考虑计算容量会导致异构集群的次优模型放置。

基于模型放置，LLM 服务需要一个能够高效服务集群中请求的请求调度策略。请求调度策略 $\phi: \mathcal{R} \mapsto \mathcal{C}^k$ 输入请求并输出一系列计算节点，这些计算节点形成执行 LLM 所有层的完整管道。

4.2 联合优化的必要性

在深入研究异构 LLM 服务的 Max-Flow 公式之前，我们首先使用一个示例来说明为什么我们需要将模型分区、设备放置和请求调度作为 Max-Flow 问题进行协同优化。在此示例中，簇如图 1a 所示。有两个区域之间的带宽较低。区域 1 拥有强大的 A100 GPU，而区域 2 拥有性能稍弱的 L4 GPU 和 3 个 T4 GPU，但该区域内的带宽较高。成对带宽是独立的。如果我们遵循常见的方法，即对模型进行静态分区，然后将设备分配给每个分区，则布局计划将如图 1b 所示。在这个计划中，虽然最后一个管道阶段有一个 T4 和一个 L4 GPU，但它的吞吐量受到前一个阶段的输出吞吐量的限制，而前一个阶段只有 2 个 T4 GPU。这表明有必要共同优化管道分区计划和管道阶段的放置。

然而，即使每个管道阶段的计算能力完全平衡，如图 1c 所示，解决方案仍然可能不是最优的。在此解决方案中，它分配强大的 A100 单独服务某些层，而其他 GPU 以数据并行方式运行其余层。然而，从一个管道阶段到另一个管道阶段的通信成为瓶颈。对于每个请求，其中间状态通过低带宽从区域 1 发送到区域 2。这最终会在 A100 的发送端造成拥塞。相反，图 1d 分配了两个与 A100 并行运行的 T4 GPU。这会在 A100 GPU 和两个 T4 GPU 之间分配工作负载，从而减少慢速链路上的通信。

4.3 异构 LLM 作为最大流

为了优化模型放置，Helix 需要一种方法来确定不同模型放置的最大服务吞吐量。为了实现这一目标，我们将具有指定模型层的计算节点集群转换为具有边缘容量的有向图。边容量表示边的数量

表 5. MILP 中使用的变量。

Symbol	Type	Num.	Description
s_i	int	$O(C)$	index of c_i 's first layer
b_i^j	binary	$O(C)$	whether c_i holds j layers
$f_{i,j}$	real	$O(\mathcal{E})$	flow from c_i to c_j
$d_{i,j}$	binary	$O(\mathcal{E})$	whether (c_i, c_j) is valid
$\text{cond}_{i,j}^1$	binary	$O(\mathcal{E})$	aux. variable in constraint-4
$\text{cond}_{i,j}^2$	binary	$O(\mathcal{E})$	aux. variable in constraint-4

令牌计算节点和网络连接每秒可以处理/传输。图中源顶点和汇顶点之间的最大流量（代表协调器节点）为我们提供了当前模型放置的集群的最大服务吞吐量。下图展示了正式的构建。

对于具有协调器节点 h 、一组计算节点 C 和模型放置 Ψ 的给定集群，我们可以将集群中的实体转换为其图形抽象的元素，如下所示。图 2 显示了具有给定模型放置的集群的图形抽象示例。

计算节点和协调节点。对于每个计算节点 $c_i \in C$ ，我们用图中的两个连接的顶点来表示它。我们将两个顶点命名为 c_i^{in} 和 c_i^{out} 。有向边 $(c_i^{\text{in}}, c_i^{\text{out}})$ 的容量表示该节点一秒内可以处理的最大令牌数。它是节点计算和网络吞吐量的最小值。Helix 执行一次性分析来测量所有计算节点的吞吐量。对于协调器节点，我们将其表示为图中的源顶点和汇顶点。

网络连接。在给定的集群中，节点可以与任何其他节点进行通信，从而在不同节点之间创建 $O(|C|^2)$ 可能的定向网络连接。然而，根据如下所述的模型放置，这些连接中只有一部分是有效的。有效的连接应满足以下三个标准之一：（1）连接是从协调器节点 h 到计算节点 c_i ，并且 c_i 保存模型的第一层；（2）连接是从计算节点 c_j 到协调器节点 h ，并且 c_j 保存模型的最后一层；（3）连接是从一个计算节点 c_i 到另一计算节点 c_j 的连接，并且 c_j 保存在推理 c_i 后立即需要的模型层。对于第一种和第二种情况，我们分别表示与有向边 $(\text{source}, c_i^{\text{in}})$ 和 $(c_j^{\text{out}}, \text{sink})$ 的连接，其容量等于连接带宽除以令牌的传输大小（几个字节）。对于第三种情况，我们用有向边 $(c_i^{\text{out}}, c_j^{\text{in}})$ 表示连接，容量等于连接带宽除以激活的传输大小（数十千字节）。Helix 执行一次性分析并使用平均带宽作为连接带宽。边缘的容量对不同节点之间的网络连接速度所施加的吞吐量约束进行建模。我们用 $\mathcal{E}$ 表示完整的可能网络连接集。

表 6. MILP 中使用的约束。

Group	Num.	Constraint
Model placement	$O(C)$ $O(C)$	$\sum_{j=1}^k b_i^j = 1$ $0 \leq s_i < L$ and $e_i \leq L$
Flow conservation	$O(C)$	$\sum_u f_{u,i} = \sum_v f_{i,v}$
Infer. throughput	$O(C)$	$\sum_u f_{ui} \leq \sum_{j=1}^k b_i^j \cdot T_j$
Connection validity	$O(C)$ $O(C)$ $O(E)$ $O(E)$ $O(E)$	$s_i \leq L \cdot (1 - d_{source,i})$ $L \cdot d_{i,sink} \leq e_i$ $(L+1)(1 - cond_{i,j}^1) \geq s_j - e_i$ $e_j - e_i \geq 1 - (L+1)(1 - cond_{i,j}^2)$ $d_{i,j} \leq 0.5 * cond_{i,j}^1 + 0.5 * cond_{i,j}^2$
Trans. throughput	$O(E)$	$f_{i,j} \leq d_{i,j} \cdot S_{i,j}$

在构建了集群的等效图抽象之后，我们运行预流推送算法[8]来获得源节点和汇节点之间的最大流量。这里的一个流量单位代表一秒内可以通过计算节点或网络连接的一个令牌。因此，最大流量为我们提供了当前模型放置的集群的最大可能服务吞吐量。

4.4 MILP 的最佳模型放置

上一节介绍了一种获取给定模型放置的集群最大服务吞吐量的方法。在本节中，我们介绍一种基于混合整数线性规划（MILP）的方法来找到最大化最大流量的模型放置，从而最大化服务吞吐量。MILP 公式具有线性数量的变量和关于计算节点和网络连接数量的约束。解决的关键挑战包括（1）将系统级约束表述为要满足的线性数量的条件，（2）用线性数量的变量表达这些条件，以及（3）使用辅助变量对每个条件进行线性化，具体来说，每个约束在最多两个辅助变量的帮助下表示为最多三个线性约束。表 5 和表 6 显示了变量和约束的概述。

节点变量。为了表示每个计算节点上的模型放置，我们在 MILP 公式中引入了两组变量。假设模型总共有 L 层，并且每个计算节点保存模型的连续子集。对于每个计算节点 c_i ，我们引入一个整数变量 s_i 来表示第一层 c_i 。假设计算节点 c_i 在其 GPU 上最多可以容纳 k 层，我们进一步引入 k 二进制变量 $b_i^1, b_i^2, \dots, b_i^k$ 来指示节点 c_i 容纳的层数（如果 c_i 容纳 j 层，则 $b_i^j = 1$ ）。我们选择使用 k 二元变量（而不是层数的一个整数）来表达模型放置，因为这一公式有助于表达推理吞吐量约束，如下所述。这

c_i 的结束层索引可以表示为 $e_i = s_i + \sum_{j=1}^k j \cdot b_i^j$ 。因此， c_i 保留范围 $[s_i, e_i]$ 中的层。连接变量。我们引入两组变量来限制可以通过每个网络连接的推理请求的数量。对于计算节点 c_i 和 c_j 之间的网络连接，我们引入一个实变量 $f_{i,j}$ 来表示图抽象中从 c_i^{out} 到 c_j^{in} 的流量。我们进一步引入一个二进制变量 $d_{i,j}$ 来表示网络连接是否有效（如 4.3 节中定义）。我们下面介绍的约束将使用 $d_{i,j}$ 来确保请求只能通过有效的连接传输。对于协调器节点和计算节点之间的网络连接，我们同样引入与上面类似的两个变量，但将 i/j 替换为 source/sink。

约束1：模型放置。为了确保 MILP 求解器找到的模型放置有效，我们需要为每个计算节点 c_i 设置以下两个约束。首先， c_i 应该只有一个有效的模型放置，即 $\sum_{j=1}^k b_i^j = 1$ 。此外，第一层和最后一层 c_i 必须在 L 层的范围内，即 $0 \leq s_i < L$ 和 $e_i \leq L$ 。（ c_i 保存范围 $[s_i, e_i]$ 内的层）

约束2：流量守恒。对于每个计算节点 c_i ，由于流量守恒，进入 c_i^{in} 的流量总和必须等于流出 c_i^{out} 的流量总和。该约束可以表示为 $\sum_u f_{u,i} = \sum_v f_{i,v}$ ，其中 u 和 v 枚举了除 i 之外的所有节点。

约束3：推理吞吐量。对于计算节点 c_i ，经过 (c_i^{in}, c_i^{out}) 的流量不应超过其最大推理吞吐量。我们可以用 $\sum_u f_{ui} \leq \sum_{j=1}^k b_i^j \cdot T_j$ 施加这个约束。这里， T_j 是一个常量，表示节点 c_i 在持有 j 层时一秒钟可以处理的最大令牌数量，这是通过一次性分析过程获得的。

约束4：连接有效性。我们需要判断网络连接的有效性，以了解请求是否可以通过它们传输。对于从协调器节点到计算节点 c_i 的网络连接，仅当 c_i 保存模型的第一层时才有效。为了用 MILP 表达这个约束，我们需要将其线性化为以下形式： $s_i \leq L \cdot (1 - d_{source,i})$ 。类似地，对于从计算节点 c_i 到协调器的网络连接，我们用 $L \cdot d_{i,sink} \leq e_i$ 约束其有效性。对于计算节点 c_i 到 c_j 的网络连接，其有效性 $d_{i,j}$ 由 $s_j \leq e_i < e_j$ 是否成立来确定。为了线性化这个条件，我们需要引入两个二元辅助变量 $cond_{i,j}^1$ 和 $cond_{i,j}^2$ 。仅当 $s_j \leq e_i$ 时 $cond_{i,j}^1$ 才取值 1，可以线性化为 $(L+1)(1 - cond_{i,j}^1) \geq s_j - e_i$ 。仅当 $e_i < e_j$ 时 $cond_{i,j}^2$ 才取值 1，可以线性化为 $e_j - e_i \geq 1 - (L+1)(1 - cond_{i,j}^2)$ 。仅当两个二进制辅助变量都为 true 时，网络连接才有效，可以表示为 $d_{i,j} \leq 0.5 * cond_{i,j}^1 + 0.5 * cond_{i,j}^2$ 。

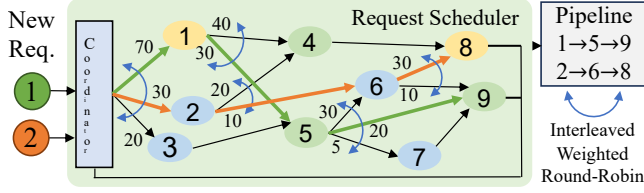


图 4. 集群的拓扑图，其中每个顶点都是一个计算节点，每条边都是一个有效的网络连接。边上的数字表示最大流量解决方案中网络连接上的流量。用于调度请求 1 和 2 的管道如右侧所示。

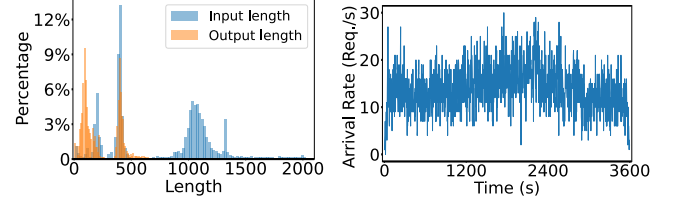
5 螺旋运行时

本节讨论 Helix 中请求的运行调度。当协调器节点收到新请求时，它会运行 Helix 的请求调度程序，为该请求分配一个每请求管道，我们将在第 2 节中介绍。5.1. 然后协调器节点将请求发送到管道中的第一个计算节点。当计算节点收到请求时，它会使用它在该管道中负责的层对请求进行推理，并将请求发送到后续计算节点。图 3 显示了 Helix 的概述。

5.1 调度程序设计：每个请求的管道

为了推断集群中的请求，调度程序需要为该请求分配管道。该管道包含一系列阶段，其中每个阶段指定一个计算节点以及在计算节点上推断的层。当顺序运行各阶段时，有效的管道必须准确地推断模型的每一层一次并按正确的顺序。现有的工作 [5, 19] 使用固定管道，其中每个管道包含一组不相交的机器，并将请求分配给这些管道。在 Helix 中，我们没有使用固定管道，而是提出了一种按请求管道分配方法，其中每个请求都有自己的管道，并且管道可以相互交叉。可能的管道总数等于集群的图形抽象中从源到汇的可能路径的数量。丰富的管道数量使得调度能够更好地适应计算节点和网络连接的容量。我们的最大流量公式使我们能够创建每个请求的管道。

Helix 请求调度器根据集群的拓扑图进行调度（图 4）。在拓扑图中，顶点对应于集群中的节点。有向边对应于有效的网络连接（在通过求解第 4 节中的 MILP 找到的模型放置下）。我们将交错加权循环（IWRR）[51] 调度程序绑定到每个顶点。IWRR 调度程序将候选列表及其权重作为输入。为了安排请求，它会选择一个与其权重成比例的频率的候选者。对于每个顶点 u ，IWRR 调度程序的候选者包含所有顶点 v ，使得有向边 (u, v) 存在（即 (u, v)



(a) Length distribution. (b) Arrival rate.

图 5. Azure 对话数据集的统计信息。

代表有效的网络连接)。 v 的权重等于最大流量解中 (u, v) 网络连接上的流量。使用 IWRR 允许我们按照最大流量安排请求，而不会产生突发。

Helix 的请求调度程序在协调器节点上运行。当新请求到达时，它首先使用代表协调器节点的顶点的 IWRR 调度程序来确定第一个管道阶段的计算节点 c_1 。然后，它使用代表 c_1 的顶点的 IWRR 调度器来确定第二阶段的计算节点 c_2 ，并重复此过程直到建立有效的管道。图 4 显示了两个请求的调度。设置请求的管道后，协调器节点将请求发送到管道中的第一个计算节点以开始推理。在推理过程中，Helix 采用动态批处理策略，其中每个节点包含前一批处理过程中收到的所有请求，以形成新的一批。这种尽力批处理的发生无需额外的等待期。

5.2 KV-缓存估计

当服务 LLM 时，每个 GPU 都有有限数量的 VRAM 来存储推理期间请求的 KV 缓存。如果在 GPU 上同时运行的请求需要的 KV 缓存超过此限制，则执行引擎必须将一些请求卸载到主内存，这会严重损害吞吐量。然而，我们并不确切知道每个请求将使用多少 KV 缓存，因为在推理完成之前输出的长度是未知的。因此，在调度程序中，我们使用平均输出长度来估计所有计算节点的 KV 缓存使用情况，并在运行 IWRR 时屏蔽掉超出高水位线的计算节点。仅当当前在这些节点上运行的某些请求完成后，我们才能向计算节点调度更多请求。这个掩码确保我们不会过度订阅 GPU 的 KV 缓存。

6 评价

在本节中，我们旨在回答以下问题。

- Helix 能否为单个集群中的异构 GPU 提供更高的吞吐量？（第 6.3 节）
- Helix 是否会牺牲延迟来换取吞吐量？（第 6.3 节）
- Helix 能否为地理分布式集群中的异构 GPU 提供更高的吞吐量？（第 6.4 节）

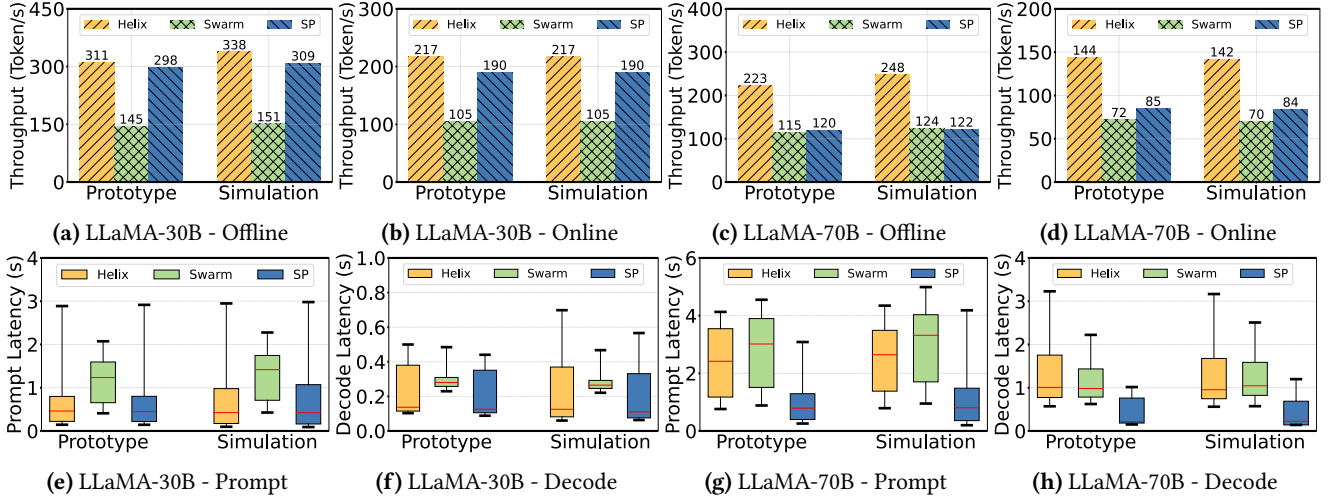


图 6. 单集群结果: (a - d) 解码吞吐量, (e - h) 在线服务的提示和解码延迟。该框显示 25 和 75 百分位。胡须显示第 5 个和第 95 个百分位。红线显示中位数。

表 7. 亚洲东部、美国中部、欧洲西部和澳大利亚东南部机器之间的网络带宽。在 Google Compute Engine 上使用 iperf3 进行测量。

Receiver	Sender			
	asia-east2-a	us-central1-f	eu-west3-c	au-se1-c
asia-east2-a	/	123 Mbps	67 Mbps	175 Mbps
us-central1-f	122 Mbps	/	204 Mbps	123 Mbps
eu-west3-c	61 Mbps	196 Mbps	/	54 Mbps
au-se1-c	159 Mbps	118 Mbps	63 Mbps	/

- 当 GPU 异构程度增加时, Helix 能否提供持续的高吞吐量? (第 6.5 节)
- 与现有系统相比, 为什么 Helix 能够实现更好的性能 (第 6.6 节和 6.7 节)?

6.1 实施

螺旋原型。我们在 Python 中实现了一个具有 1.5k LoC 的多副本管道并行系统, 在 C++ 中实现了具有 1.7k LoC 的多副本管道并行系统。对于模型执行, 我们采用最新版本的 vLLM [22] (0.4.0post1) 以避免重新实现基本的 LLM 推理优化。我们在 vLLM 之上实现了统一的页面池来支持部分推理。我们使用 ZeroMQ [52] 进行节点间通信, 使用 Gurobi [14] 作为 MILP 求解器。模拟器。我们还在 Python 中实现了一个具有 14k LoC 的分布式 LLM 推理模拟器。它支持异构 GPU 和网络条件的模拟。它使我们能够灵活地探索更多样化的网络、GPU 异构性和集群规模设置。秒。6.3 评估了模拟器的保真度, 结果显示所有指标的模拟误差均小于 5%。

6.2 实验设置

模型。我们在 LLaMA [55, 56] 上评估 Helix, LLaMA 是一个具有代表性且流行的开源 Transformer 模型系列。

具体来说, 我们使用 LLaMA-1 30B 和 LLaMA-2 70B 来研究不同尺寸模型上的系统性能。我们以半精度 (FP16) 运行模型推理。在后续部分中, 我们将这两个型号称为 LLaMA 30B 和 LLaMA 70B。

集群设置。我们使用三种集群设置进行评估: (1) 单集群 (第 6.3 节)、(2) 地理分布式集群 (第 6.4 节) 和 (3) 高 GPU 异构性集群 (第 6.5 节)。首先, 单个集群设置包含通过 10Gb/s 网络连接的 4 个 A100 节点、8 个 L4 节点和 12 个 T4 节点。我们将网络带宽配置为 10 Gb/s, 因为该速率足以确保 LLM 服务受到 GPU 吞吐量而不是网络容量的限制。我们在 Google Cloud 上的一个区域内分配节点。其次, 地理分布式集群包含三个集群, 其中 (i) 4 个 A100 节点, (ii) 2 个 L4 节点+8 个 T4 节点, 以及 (iii) 6 个 L4 节点+4 个 T4 节点。集群间通信的平均带宽为 100Mb/s, 平均延迟为 50ms。根据表 7 中的分析结果, 我们将带宽配置为 100 Mb/s 来模拟跨区域网络限制。最后, 高 GPU 异构集群包含 4 个 A100 节点、6 个 V100 节点、8 个 L4 节点、10 个 T4 节点、4 个 2×L4 节点、6 个 2×T4 节点和 4 个 4×T4 节点。后两种设置是在模拟中执行的。我们对原型系统的评估侧重于单 GPU 节点, 因为多 GPU 节点在云中分配起来要困难得多 [53]。我们的实现也适用于多 GPU 节点, 通过利用 vLLM [22] 支持的同一节点上 GPU 之间的张量模型并行性。

痕迹。我们使用的跟踪来自 Azure 对话数据集 [40]。图 5 显示了该数据集的长度分布和到达率。我们删除输入长度大于 2048 或输出长度大于 1024 的请求, 以维持 vLLM 合理的运行时内存使用量。

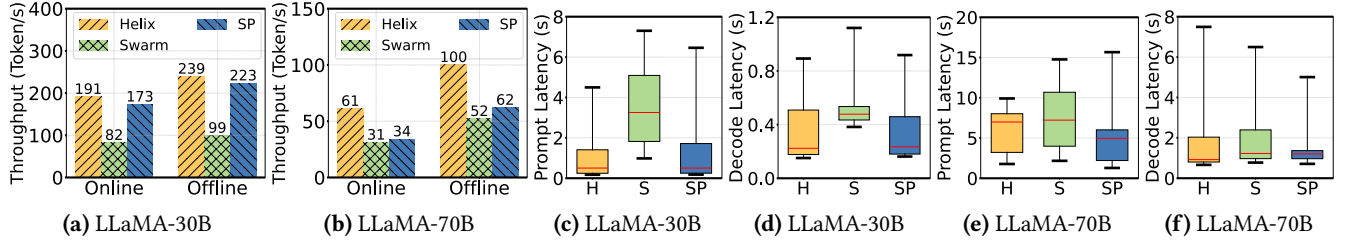


图 7. 地理分布式集群: (a - b) 解码吞吐量, (c - f) 在线服务的提示和解码延迟。“H”代表Helix, “S”代表Swarm。方框: 25 和 75 百分位。晶须: 5% 和 95%。红线: 中位数。

修剪后的数据集包含 16657 个请求, 平均输入长度为 763, 平均输出长度为 232。我们注意到, 由于请求长度较长, 该数据集比先前工作 [22] 使用的数据集更具挑战性。我们两种到达率设置。对于在线设置, 我们使用 Azure 对话数据集的实际到达率。我们将平均到达率扩展到集群峰值吞吐量的 75%, 以避免突发请求导致系统 OOM。对于离线设置, 我们允许请求达到充分利用集群所需的速率。这模拟了在数据集上运行离线推理。我们将这两种设置称为在线服务和离线服务。

实验持续时间。对于在线设置, 我们将集群预热 30 秒并测试 30 分钟。对于离线设置, 我们预热集群 1 分钟并测试 10 分钟。这个时间对于我们的评估来说是足够的, 我们不会跑更长的时间来减少不必要的实验成本。

螺旋设置。我们允许 MILP 求解器使用和不使用部分推理和聚类修剪进行搜索。我们还使用 Petals/Swarm/单独管道的解决方案提示 MILP 求解器。当 MILP 求解器在 10 分钟内找不到更好的解时, 求解就会超时。每个集群设置的最大搜索预算在 14 核 CPU 上为 4 小时。

基线。据我们所知, 不存在适用于我们设置的模型放置和调度的异构 LLM 服务系统。因此, 我们采用异构 LLM 培训系统 Swarm [45] 的思想来构建有竞争力的异构基线。我们还构建了另一个基线, 通过为每种类型的机器提供一个模型副本来处理 GPU 异构性。我们将这两个基线称为 Swarm 和分离管道 (SP)。为了确保公平比较, 我们在我们的系统中实施了 SP 基线, 而不是利用其他系统。我们进一步与第 2 节中 Petals [5] 的模型放置进行比较。6.6, 这是一个用于志愿者计算的去中心化 LLM 服务系统。我们不会在端到端服务方面与 Petals 进行比较, 因为它缺乏集中的请求调度。对于 Swarm, 我们在我们的系统中实现了他们的模型放置和调度算法, 因为他们的原始系统不能用于推理。我们将管道级数设置为最小值, 允许最弱的 GPU 用一半的 VRAM 来容纳一个级。这

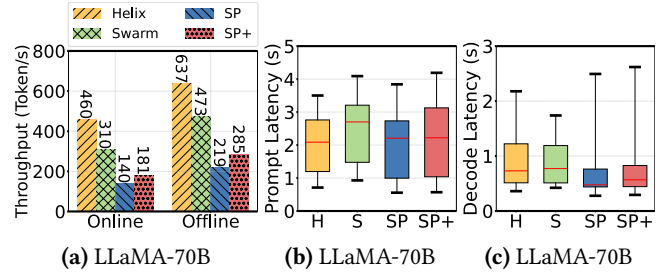


图 8. 高 GPU 异构性集群设置: (a) 解码吞吐量, (b - c) 在线服务的提示和解码延迟。“H”代表Helix, “S”代表Swarm。

最大限度地减少管道深度并为 KV 缓存留下足够的 VRAM, 这两者对于性能都至关重要。指标。对于离线服务, 我们报告平均解码吞吐量, 即每秒生成的令牌数量。对于在线服务, 我们进一步报告平均提示延迟和解码延迟, 分别是解析用户输入和生成新令牌的平均延迟。

6.3 单集群

本部分通过在单集群设置中提供 LLaMA 30B 和 70B 在线和离线服务来评估 Helix。结果如图 6 所示。对于 LLaMA 30B, 每种 GPU 类型都有足够的节点来服务至少一个单独的管道。Helix 发现的最佳模型放置服务于模型的三个副本, 每个副本都具有一种类型的 GPU。因此, Helix 和 SP 实现了相似的性能: 由于其更好的 KV 缓存利用率, Helix 的离线和在线服务解码吞吐量分别高出 4% 和 14%; 提示延迟降低 2%, 解码延迟提高 10%。与 Swarm 相比, Helix 的离线和在线服务解码吞吐量分别提高了 2.14× 和 2.07×。Helix 还将在在线服务的提示和解码延迟分别减少了 32% 和 12%。我们发现 Swarm 的模型放置引入了瓶颈并且未充分利用 A100 节点, 我们将在第 2 节中更详细地讨论这一点。6.6. 我们注意到, 与其他 LLM 服务系统 [22, 40] 相比, 我们的实验中观察到的延迟相对较高, 这可归因于我们使用功能较弱的 GPU (T4 和 L4), 而其他研究中使用的是 A100 和 H100 GPU。

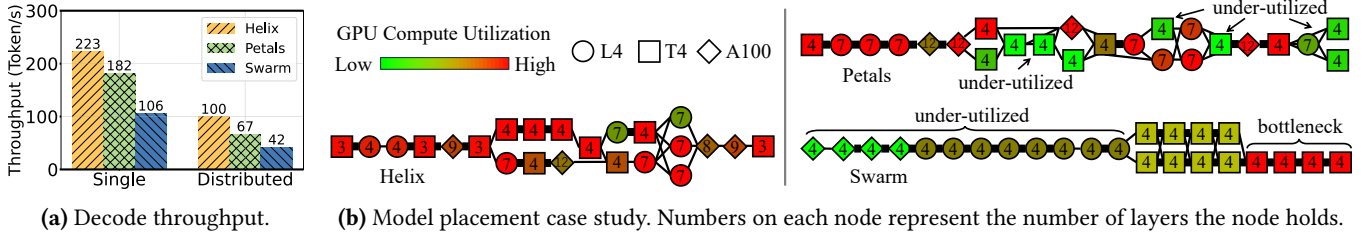


图 9.(a) 将不同模型放置方法与 LLaMA 70B 离线服务进行比较。(b) 案例研究显示了在单集群设置上为 LLaMA 70B 提供服务的模型布局。

对于 LLaMA 70B，来自单一 GPU 类型的节点无法单独为一个模型副本提供服务，同时为 KV 缓存留下足够的 VRAM。在这种情况下，SP 的吞吐量显著下降。与 SP 相比，Helix 在离线和在线服务中实现了 $1.86\times$ 和 $1.69\times$ 高的解码吞吐量。Helix 的平均延迟高于 SP，因为 SP 通过 A100 节点服务大部分请求，而 L4 和 T4 节点利用率不足。与 Swarm 相比，Helix 在离线和在线服务中实现了 $1.94\times$ 和 $2.00\times$ 高的解码吞吐量。Helix 的提示延迟降低了 15%，但由于 Helix 的 GPU 利用率较高，解码延迟增加了 16%。与 LLaMA 30B 类似，Swarm 由于其模型放置而未充分利用 A100 节点。

我们还在模拟器中进行这些实验（参见第 6.1 节）以检查其保真度。仿真结果与实际系统评估结果一起如图 6 所示。解码吞吐量的平均误差低于 5%。提示和解码延迟的平均误差分别低于 5% 和 4%。这表明我们的模拟器实现了高保真度，并且可以用于比较不同方法的性能。

6.4 地理分布式集群

本节使用我们在第 2 节中的高保真模拟器来评估 Helix 在地理分布式集群中在线和离线服务 LLaMA 30B 和 70B 的能力。6.1 图 7 显示了结果。尽管此设置涉及与单个集群相同的 GPU 集，但缓慢的集群间网络会导致所有方法具有较低的吞吐量和较高的延迟。

对于 LLaMA 30B，Helix 发现的最佳模型放置仍然由分别由三种类型的机器提供服务的三个管道组成。与 SP 相比，Helix 的离线和在线服务解码吞吐量分别提高了 7% 和 10%。提示延迟降低了 14%，解码延迟提高了 2%。与 Swarm 相比，Helix 在线和离线服务的解码吞吐量分别提高了 $2.41\times$ 和 $2.33\times$ 。Helix 还将在线服务的提示和解码延迟减少了 66% 和 24%。

另一方面，由于更大的激活大小和模型深度，服务 LLaMA 70B 对慢速网络更敏感。Helix 通过使用管道深度较浅的模型放置来减少网络开销。模型放置

Helix 发现，与 Swarm 相比，管道深度减少了 28%。当网络速度很快时，它也比 Helix 发现的浅 19%。Helix 的模型放置规划器平衡网络开销与单节点 GPU 利用率，在地理分布式 GPU 集群中实现高吞吐量和低延迟。与 SP 相比，Helix 的离线和在线服务解码吞吐量分别提高了 $1.61\times$ 和 $1.79\times$ 。SP 具有较低的平均延迟，因为它使用快速 GPU 满足大多数请求，而 12 个 T4 节点由于 VRAM 限制而未得到充分利用。与 Swarm 相比，Helix 的离线和在线服务解码吞吐量分别提高了 $1.92\times$ 和 $1.97\times$ 。Helix 还将提示和解码延迟减少了 21% 和 7%。我们观察到 Swarm 离线服务期间出现严重拥塞。平均提示延迟达到 71s，是 Helix 的 $7.5\times$ 。我们将在第 2 节中通过案例研究更多地讨论 Swarm 的拥塞问题。6.7。

6.5 高 GPU 异构集群

本节在具有 42 个计算节点和 7 种 GPU 的高度异构 GPU 集群上评估 Helix。图 8 显示了结果。在该集群中，V100、T4 和 T4x2 节点无法自行形成服务管道。我们报告没有这些机器的 SP 的吞吐量。我们还尝试使用这些机器构建混合管道，并将混合管道的数量报告为 SP+。与 Swarm、SP 和 SP+ 相比，Helix 的离线服务吞吐量为 $1.37\times$ 、 $2.91\times$ 和 $2.24\times$ ，在线服务吞吐量为 $1.48\times$ 、 $3.29\times$ 和 $2.54\times$ 。Helix 还分别将提示延迟减少了 17%、1% 和 7%。Helix 的平均解码延迟稍高，因为基线未充分利用慢速 GPU，并使用具有大 VRAM 的快速 GPU 来满足大多数请求。这一结果表明，当集群中存在多种类型的 GPU 时，Helix 可以实现一致的高性能。

6.6 模型放置深入探讨

在本节中，我们分析不同模型放置方法对集群服务吞吐量的影响。我们评估了 Helix 在单个集群和地理分布式集群上的离线服务性能，并将 Helix 的模型放置方法与 Swarm 和 Petals 中使用的模型放置方法进行了比较。

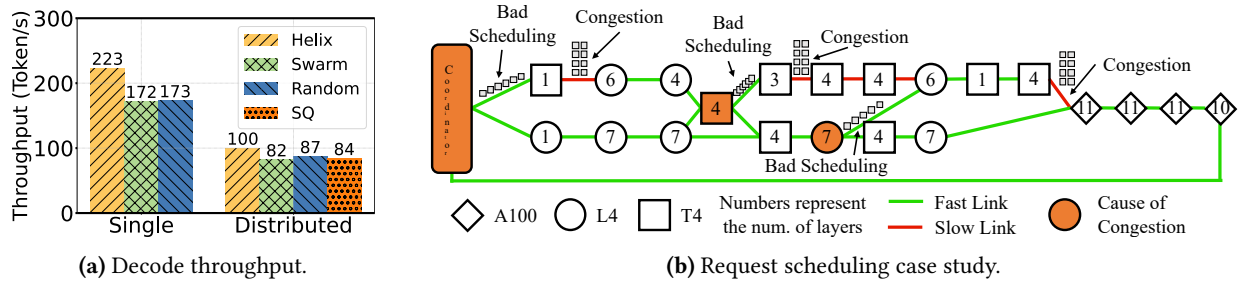


图 10.(a) 将不同的请求调度方法与 LLaMA 70B 的离线服务进行比较。(b) 案例研究说明了在地理分布式集群设置中为 LLaMA 70B 提供服务时 Swarm 和随机调度中的拥塞情况。

两种方法都使用基于吞吐量的启发法来执行模型放置。为了隔离模型放置的影响，我们对所有方法使用 Helix 的请求调度程序。图 9a 显示了解码吞吐量。与 Petals 和 Swarm 相比，Helix 在单个集群上的吞吐量提高了 1.23× 和 2.10×，在地理分布式集群上的吞吐量提高了 1.49× 和 2.38×。我们对 LLaMA 70B 进行案例研究，以证明 Helix 为何能实现最佳性能。

案例研究：LLaMA 70B - 单簇。图 9b 显示了在单个集群上提供 LLaMA 70B 时每种方法的模型布局和 GPU 计算利用率。Swarm 的模型放置在其管道末端引入了瓶颈，其中 4 个 T4 节点每个服务于 4 个层。此瓶颈导致 A100 和 L4 节点上的 GPU 利用率不足，从而显著降低服务吞吐量。对于 Petals 的模型放置，8 个 T4 节点和 1 个 L4 节点未得到充分利用，这对服务吞吐量产生了负面影响。对于 Helix 的模型放置，几乎所有节点都被充分利用。GPU 的高效使用使 Helix 的性能分别比 Swarm 和 Petals 高出 2.10× 和 1.23×。

6.7 请求调度深入探讨

本节分析不同请求调度方法对集群服务吞吐量的影响。我们评估了 LLaMA 70B 在单集群和地理分布式集群上的 Helix 离线服务性能。我们将 Helix 的请求调度器与 (1) Swarm 进行比较，Swarm 根据每个候选节点的实时吞吐量来调度请求，以及 (2) 随机调度，在调度中随机选择一个候选节点。对于地理分布式集群设置，我们进一步与最短队列优先调度 (SQ) 进行比较，它始终将请求分配给具有最短队列的节点。为了消除模型放置的影响，所有方法都使用 Helix 找到的模型放置。图 10a 显示了解码吞吐量。与 Swarm 和随机调度相比，Helix 在单个集群上的吞吐量分别提高了 30% 和 29%，在地理分布式集群上的吞吐量分别提高了 22% 和 15%。与最短队列优先调度相比，Helix 的解码吞吐量提高了 19%。此外，运行时监控

表 8. 有和没有修剪的问题大小。var 表示变量，cstr 表示约束。

Problem size	With pruning	Without pruning
24-node	876 var 1122 cstr	1376 var 1848 cstr
42-node	2144 var 2772 cstr	4004 var 5502 cstr

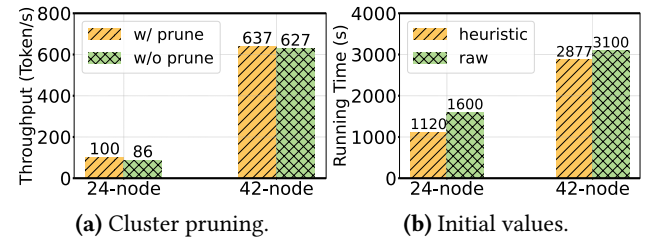


图 11. MILP 优化的消融研究。

表明所有三种基线调度方法都会引入严重的拥塞。我们在案例研究中进一步说明了这一点。案例研究：LLaMA 70B - 分布式集群。图 10b 显示了 Helix 发现的用于在地理分布式集群上提供 LLaMA 70B 服务的模型放置计划。该计划尽可能避免使用慢速的集群间网络连接，但少数计算节点仍然采用低带宽连接。当使用 Swarm 或随机调度来调度请求时，我们观察到图中标记为“拥塞”的三个链路上出现严重拥塞——提示阶段请求在这些链路上排队平均需要 5 秒至 16 秒才能传输。我们找出造成拥塞的节点的根本原因，并在图中将它们标记为橙色。令人惊讶的是，我们发现一次拥塞是由 3 跳之外的节点的错误调度引起的。这验证了一种能够兼顾网络和计算的全局调度方法的必要性。当使用 Swarm 的请求调度为地理分布式集群找到的模型放置提供 LLaMA 70B 服务时，我们还观察到类似的拥塞。

6.8 优化消融研究

本节对第 2 节中介绍的两种 MILP 优化进行了消融研究。4.5.我们评估了 LLaMA 70B 在地理分布式和高度分布式上的离线服务。

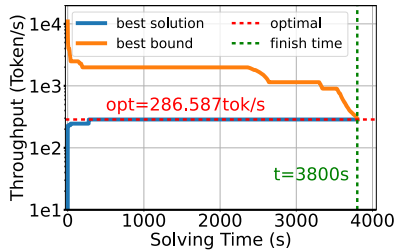


图 12. MILP 求解器找到的相对于求解时间的最佳解决方案和最佳上限。红色虚线标记了该集群的最佳吞吐量。

异构集群，分别称为 24 节点和 42 节点设置。

集群修剪。启用集群修剪后，我们会修剪网络连接，使每个节点的平均度数为 12，这对于我们下面讨论的 LLM 推理系统来说已经足够了。启用集群修剪会删除 24 和 42 节点设置的 50% 和 72% 的网络连接。图 8 显示，对于两种设置，修剪将问题大小分别减少了 36% 和 46%。图 11a 显示，当使用通过集群修剪发现的模型放置时，Helix 的解码吞吐量分别提高了 16% 和 2%。我们注意到，所实现的加速量会根据当前 MILP 问题的具体实例而有所不同。修剪慢速网络连接不会损害吞吐量，因为服务中使用的网络连接非常稀疏——通常每个节点只与少数其他节点通信。此外，还有许多等效的模型布局可以实现相同的吞吐量。修剪集群很可能会使其其中一些放置仍然有效。随着问题规模（和解决方案空间大小）的减小，它使得在有限的优化时间内搜索这些位置变得更加容易。

初始值。我们从启发式方法的解决方案和默认值开始比较运行 Helix 模型放置规划器的性能。由于找到的最佳模型放置位置相同，因此我们比较挂钟时间来找到放置位置。图 11b 显示，对于 24 节点和 42 节点设置，从启发式解决方案运行 MILP 所需的时间分别减少了 43% 和 8%。我们注意到，所实现的加速会根据当前 MILP 问题的具体实例而有所不同。结果表明，从启发式解决方案开始可以加速 Helix 中的模型放置。

6.9 模型放置质量

本节评估 Helix 相对于 MILP 求解时间找到的模型布局的质量。在此评估中，我们运行 Helix 来找到在具有 4 个 L4 和 6 个 T4 机器的集群中为 LLaMA 30B 提供服务的最佳模型放置。我们记录了最佳模型放置以及我们使用的 MILP 求解器 (Gurobi) 找到的最佳上限

求解过程。最佳上限代表 MILP 问题可以实现的最佳目标值，并且随着求解器探索更多节点并添加切割平面，该上限将逐渐变得更紧。图 12 显示了最佳模型放置的质量和相对于求解时间的最佳上限。结果表明，Helix 在不到 5 分钟的时间内找到了最优解，但求解器需要一个多小时才能枚举所有可能的解并确认最优性。这表明我们可以提前停止求解过程，因为高质量的解决方案会在计算的早期阶段出现。

7 相关工作

机器学习模型服务有大量的工作用于服务机器学习模型，讨论的方面包括系统实现[36,37]、模型放置[24,41,48,58]、请求调度[13,48,63]和尾部延迟缓解[20,21,31]。然而，由于 LLM 独特的自回归执行范式，这些方法无法有效地服务于 LLM。相反，许多最近的 LLM 特定系统解决了 LLM 服务中不可预测的执行时间和高内存消耗问题。Orca [61]提出了迭代级调度，以便在请求完成后释放资源。vLLM [22]引入了 PageAttention，通过分配其所需的确切页面数来进一步减少每个请求的内存消耗。推测推理 [23, 30] 应用一个小模型来预测多个输出标记，并在单次迭代中验证它们。Splitwise [40] 和 DistServe [65] 发现，分解提示和解码阶段可以提高吞吐量，因为这两个阶段具有不同的工作负载特征。Sarathi [2] 引入了分块预填充，它将预算分配给提示阶段，以使每个微批次的工作负载平衡，从而最大限度地减少管道泡沫。所有上述工作都与我们的工作正交，并且可以集成到我们的系统中，因为我们的重点是集群异构性。

异构集群上的 ML 工作负载。有几种方法利用异构 GPU 来执行 ML 任务。其中一些 [18, 39] 共同设计模型分区并放置在异构集群上，但假设网络带宽统一。Learning home [46] 和 DeDLOC [9] 研究了去中心化集群上的网络感知路由，但仅单独考虑数据或管道并行性。SWARM [45]，如第 2 节中所讨论的。2.1、优化了异构网络中的管道通信。然而，它仅根据下一阶段的元数据进行调度，缺乏全局视图。还有一些努力使用近似来减少网络通信[57]或同步[16]。他们大多数都专注于模型训练。在模型推理中，尤其是大语言模型和地理分布式 GPU 的服务尚未得到很好的研究。SkyPilot [60] 和 Mélange [12] 为请求选择最佳的 GPU 类型，但是

每个请求均由单一 GPU 类型提供服务。花瓣 [5], 如第 2 节所述。2.1, 研究分散式管道并行设置。它为动态设备组设计了贪婪模型分配和请求调度, 失去了固定设备组的优化机会。HexGen [19] 是一项关于在异构集群中服务的 LLM 的并发工作。它基于固定管道 (参见第 5.1 节), 并采用启发式方法来搜索最佳模型放置。相比之下, 我们的 Max Flow 公式和按请求管道比 HexGen 更灵活, 并且我们的 MILP 公式可以保证最佳解决方案。

异构资源的调度 对于异构资源的调度算法存在着广泛的研究。例如, Linux 内核中的能量感知调度 [25] 为异构 CPU 拓扑调度任务。还有一些工作[4,43,54]关注异构集群中的通用调度。由于 LLM 推理的自回归性质, 这些方法不能直接用于异构 LLM 服务。

8 结论

本文介绍了 Helix, 这是第一个用于异构 GPU 集群的高吞吐量、低延迟 LLM 服务引擎, 具有保证吞吐量最大化的最佳解决方案。Helix 将模型放置和请求调度作为最大流问题来制定和解决。与现有解决方案相比, Helix 在吞吐量和延迟方面实现了显着改进。

致谢

我们感谢匿名审稿人和我们的牧羊人 Íñigo Goiri 提供的宝贵反馈和建设性建议, 帮助改进了本文。我们还对 Google Cloud Innovator 计划表示感谢, 感谢他们为我们的实验提供了 Google Compute Engine 上的机器。这项工作得到了斯隆基金会奖学金和 VMware 系统研究奖的部分支持。这项工作还得到了国家自然科学基金的部分支持, 拨款号为 CNS-2147909、CNS-2211882 和 CNS-22393 51, 以及亚马逊和 Meta 的礼品奖。

神器

A.1 摘要

我们的产品包括 Helix 的模拟器和原型系统, 用于在异构和地理分布式集群中提供分布式 LLM 服务。该代码实现了关键算法, 包括基于 MaxFlow 的 LLM 服务公式、基于 MILP 的模型放置规划器和每个请求的管道调度器。我们还提供全面的文档和脚本, 用于从头开始进行环境设置和系统执行。这些工件包含相同的模拟器

以及论文中 Helix 评估中使用的原型系统。模拟器可以在单机上运行, 而原型系统需要集群部署。

A.2 工件检查表 (元信息)

- 算法: 基于MaxFlow的LLM服务制定; 基于MILP的模型布局规划器; 每个请求的管道调度程序
- 编译: 模拟器是用Python实现的。原型系统的节点间通信框架是用C++(编写的, 建议使用GCC 13.2)编译, 并包含一个使用CMake的自动编译脚本, 而其其余组件则用Python实现。
- 型号: 我们使用LLaMa-2 70B作为测试工作负载。原型系统使用虚拟权重运行, 仅需要模型架构规范(在我们的代码存储库中提供)。
- 数据集: 我们使用 Azure 对话数据集作为我们的评估跟踪, 并在我们的代码存储库中包含预解析的版本。
- 运行环境: 对于模拟器, 我们推荐使用Python 3.10, 同时它也可以支持其他最新的Python版本。模拟器对操作系统版本不敏感。对于原型系统, 我们建议使用 Ubuntu 24.04 LTS 来搭建环境。我们还建议使用 conda 来隔离两个系统的运行时环境。详细的软件依赖关系请参阅第 2 节。A.3.3。
- 硬件: 模拟器在单机上运行, 没有特定的硬件要求。该原型系统在 24 台配备 NVIDIA GPU 的机器组成的集群上运行。请参阅第 2 节。A.3.2 了解详细的硬件要求。
- 指标: 我们使用与论文中提供的相同指标进行评估: 解码吞吐量、解码延迟和提示处理延迟。
- 输出: 结果记录到终端输出和文件中。我们将预期结果包含在代码存储库中。
- 需要多少磁盘空间 (大约)? : 所有日志文件的总大小约为 300 MB。
- 准备工作流程需要多少时间 (大约)? : 环境设置大约需要 1 - 2 小时。
- 完成实验需要多长时间 (大约)? : 功能评估大约需要 2 小时。再现性评估大约需要 16 小时。(需要整个24机集群的部分大约需要4个小时。即需要整个集群的部分是第6.3、6.6和6.7节的部分。)
- 公开吗? : 是的。DOI: 10.5281/zenodo.14037926。
- 代码许可证 (如果公开可用)? : Apache 2.0

A.3 描述

A.3.1 如何访问。该工件可在 <https://github.com/Thesys-lab/Helix-ASPLOS25> 上公开获取。它还存档为 10.5281/zenodo.14037926。文件大小约为 300 MB。请参阅我们的 Github 存储库以获取最新版本。

A.3.2 硬件依赖性。该模拟器在单机上运行, 没有特定的硬件要求。我们建议至少 32 GB 内存来模拟大型

集群以防止内存不足问题。原型系统需要集群部署 - 我们的示例配置使用 24 台机器，其中包括 4 台具有 1×A100-40GB 的机器、8 台具有 1×L4 的机器和 12 台具有 1×T4 的机器。我们建议将每台计算机设置为至少 16 个 CPU 核心和 128 GB 内存，以避免内存不足问题。理想情况下，机器之间的网络连接应至少为 10 Gbps，延迟约为几毫秒。通常常见云提供商的同一区域的机器就可以满足网络要求。出于功能测试的目的，也可以使用来自不同地区的机器。

A.3.3 软件依赖性。我们建议使用 Python 3.10 运行模拟器。它依赖于 networkx、matplotlib 和 gurobipy。基于 MILP 的模型布局规划器使用 Gurobi 作为 MILP 求解器。在代码库中运行示例不需要额外的许可证。但是，如果您想为更大的集群运行模型放置，则需要获取不限问题大小的 Gurobi 许可证。我们建议在 Ubuntu 24.04 LTS 和 Python 3.10 上运行原型系统。构建节点间通信框架需要安装 build-essential、cmake、libzmq、cppzmq 和 pybind 11。要运行原型系统，您需要安装 CUDA 12.6 和 vLLM 0.4.0.post1。有关设置环境和运行实验的分步指南，请参阅 <https://github.com/Thesys-lab/Helix-ASPLOS25/blob/master/readme.md>。

A.3.4 数据集。我们使用 Azure 对话数据集作为我们的评估跟踪，并在我们的代码存储库中包含预解析的版本。

A.3.5 模型。我们使用 LLaMa-2 70B 作为测试工作负载。原型系统使用虚拟权重运行，仅需要模型架构规范。

A.4 再现结果的详细步骤

我们提供了一个示例来展示我们系统的功能，请参阅 <https://github.com/Thesys-lab/Helix-ASPLOS25/blob/master/readme.md>。我们还提供了另一个示例来重现我们在论文中得到的所有结果，请参阅 https://github.com/Thesys-lab/Helix-ASPLOS25/blob/master/artifact_evaluation/ae_readme.md。

参考

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat 等。Gpt-4 技术报告。arXiv 预印本 arXiv: 2303.08774, 2023。[2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani 和 Ramachandran Ramjee. Sarathi: 通过使用分块预填充进行解码，实现高效的 1lm 推理。arXiv 预印本 arXiv:2308.16369, 2023。

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón 和 Sumit Sanghai. Gqa: 多头检查点训练广义多查询变压器模型。arXiv 预印本 arXiv:2305.13245, 2023。[4] Rashmi Bajaj 和 Dharma P Agrawal. 改进异构环境中的任务调度。IEEE Transactions on Parallel and Distributed Systems, 15(2):107–118, 2004。[5] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin 和 Colin Raffel. Petals: 大型模型的协作推理和微调。arXiv 预印本 arXiv: 2209.01188, 2022。[6] Alexander Borzunov, Max Ryabinin, Artem Chumachenko, Dmitry Baranchuk, Tim Dettmers, Younes Belkada, Pavel Samygin 和 Colin A Raffel. 通过互联网对大型语言模型进行分布式推理和微调。神经信息处理系统进展, 36, 2024。[7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell 等。语言模型是小样本学习者。神经信息处理系统的进展, 33: 1877–1901, 2020。[8] Joseph Cheriyan 和 SN Maheshwari. 最大网络流量的预流推送算法分析。SIAM 计算杂志, 18(6): 1057–1086, 1989。[9] Michael Diskin, Alexey Bukhtiyarov, Max Ryabinin, Lucile Saulnier, Anton Sinitsyn, Dmitry Popov, Dmitry V Pyrkun, Maxim Kashi, Alexander Borzunov, Albert Villanova del Moral 等。开放协作中的分布式深度学习。神经信息处理系统进展, 34: 7879–7897, 2021。[10] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, 等。骆驼 3 群模型。arXiv 预印本 arXiv:2407.21783, 2024。[11] 谷歌云。谷歌云计算产品。https://cloud.google.com/products/compute, 2024 年。访问时间: 2024 年 3 月 13 日。[12] Tyler Griggs, Xiaoxuan Liu, Jiayang Yu, Doyoung Kim, Wei-Lin Chiang, Alvin Cheung 和 Ion Stoica. M'elange: 通过利用 GPU 异构性提供具有成本效益的大型语言模型。arXiv 预印本 arXiv:2404.14527, 2024。[13] Arpan Gujarati, Reza Karimi, Safya Alzayat, Weihao, Antoine Kaufmann, Ymir Vigfusson 和 Jonathan Mace. 像发条一样服务 {DNN}: 自下而上的性能可预测性。第 14 届 USENIX 操作系统设计与实现研讨会 (OSDI 20), 第 443–462 页, 2020 年。[14] Gurobi Optimization, LLC. Gurobi 优化器参考手册, 2023。[15] Sirui Hong, Xiawu Cheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: 多智能体协作框架的元编程。arXiv 预印本 arXiv: 2308.00352, 2023。[16] Kevin Hsieh, Aaron Harlap, Nandita Vijaykumar, Dimitris Konomis, Gregory R Ganger, Phillip B Gibbons 和 Onur Mutlu. Gaia: {地理分布式}机器学习接近{LAN}速度。第 14 届 USENIX 网络系统设计与实现研讨会 (NSDI 17), 第 629–647 页, 2017 年。[17] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: 使用管道并行性对巨型神经网络进行高效训练。神经信息处理系统进展, 32, 2019。[18] 贾贤彦, 蒋乐, 王昂, 肖文聪, 石子吉, 张杰, 李新元, 陈朗石, 李勇, 郑振, 等。Whale: 通过异构 {GPU} 进行高效的巨型模型训练。2022 年 USENIX 年度技术会议 (USENIX ATC 22), 第 673–688 页, 2022 年。[19] Youhe Jiang, Ran Yan, Xiaozhe Yao, Yang Zhou, Beidi Chen 和 Binhang Yuan. Hexgen: 大型语言模型的生成推理

- 异构环境。第四十一届机器学习国际会议, 2024 年。[20] Jack Kosaian, KV Rashmi 和 Shivaram Venkataraman. 奇偶校验模型: 预测服务系统的纠删码弹性。第 27 届 ACM 操作系统原理研讨会论文集, 第 30-46 页, 2019 年。[21] Jack Kosaian, KV Rashmi 和 Shivaram Venkataraman. 基于学习的编码计算。IEEE Journal on Selected Areas in Information Theory, 1(1):227–236, 2020. [22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Shen, Lianmin Cheng, Codyhao Yu, Joseph Gonzalez, Hao Zhu 和 Ion Stoica. 通过 pagedattention 服务的大型语言模型的高效内存管理。收录于 2023 年 ACM SIGOPS 第 29 届操作系统原理研讨会论文集。[23] Yaniv Leviathan, Matan Kalman 和 Yossi Matias. 通过推测解码从 Transformer 进行快速推理。国际机器学习会议, 第 19274-19286 页。PMLR, 2023. [24] 李卓瀚, 郑联民, 钟银民, 刘文森, 盛英, 金鑫, 黄艳平, 陈志峰, 张浩, Joseph E Gonzalez, 等。{AlpaServe}: 具有模型并行性的统计多路复用, 用于深度学习服务。第 17 届 USENIX 操作系统设计与实现研讨会 (OSDI 23), 第 663–679 页, 2023 年。[25] Linux 内核文档。能源意识调度。https://docs.kernel.org/scheduler/sched-energy.html, 2024. Linux 内核文档。[26] 罗紫阳, 徐灿, 赵璞, 孙庆丰, 耿秀波, 胡文祥, 陶重阳, 马静, 林庆伟, 蒋大新。Wizard-coder: 使用 evol-instruct 为大型语言模型提供支持。arXiv 预印本 arXiv:2306.08568, 2023. [27] 梅志宇、付伟、李凯伟、王光举、张焕辰和吴一。Realhf: 通过参数重新分配优化大型语言模型的 r1hf 训练, 2024 年。[28] Meta. Meta Llama 3 简介: 迄今为止最有能力的公开大语言模型。Meta.com。https://ai.meta.com/blog/meta-llama-3/. [访问时间: 2024 年 7 月 5 日]。[29] 苗旭鹏, Gabriele Oliaro, 张志浩, 程新浩, 金宏毅, 陈天琪, 贾志浩。迈向高效的生成式大语言模型服务: 从算法到系统的调查。arXiv 预印本 arXiv:2312.15234, 2023. [30] 苗旭鹏, Gabriele Oliaro, 张志豪, 程新浩, 王泽宇, 张正新, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, 等。Specinfer: 通过基于树的推测推理和验证来加速大型语言模型。第 29 届 ACM 国际编程语言和操作系统架构支持会议记录, 第 3 卷, 第 932-949 页, 2024 年。[31] Hema Venkata Krishna Giri Narra, Zhifeng Lin, Ganesh Ananthanarayanan, Salman Avestimehr 和 Murali Annamaram. 拼贴推理: 使用编码冗余来降低分布式图像分类系统中的延迟变化。2020 年 IEEE 第 40 届分布式计算系统国际会议 (ICDCS), 第 453-463 页。IEEE, 2020. [32] NVIDIA. NVIDIA A100 张量核心 GPU。https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf。访问时间: 2024 年 10 月 28 日。[33] 英伟达。NVIDIA H100 张量核心 GPU。https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet。访问时间: 2024 年 10 月 28 日。[34] 英伟达。NVIDIA L4 张量核心 GPU。https://resources.nvidia.com/en-us-data-center-overview/l4-gpu-datasheet。访问时间: 2024 年 10 月 28 日。[35] 英伟达。NVIDIA T4 张量核心 GPU。https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf。访问时间: 2024 年 10 月 28 日。[36] 英伟达。Triton 推理服务器。
- [37] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmse, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh 和 Jordan Soyke. Tensorflow-serving: 灵活、高性能的机器学习服务。arXiv 预印本 arXiv:1712.06139, 2017. [38] OpenAI. ChatGPT, 2023. [39] Jaey H Park, Gyeongchan Yun, M Yi Chang, Nguyen T Nguyen, Seungmin Lee, Jaesik Choi, Sam H Noh 和 Young-ri Choi. {HetPipe}: 通过集成流水线模型并行性和数据并行性, 在 (奇特的) 异构 {GPU} 集群上启用大型 {DNN} 训练。2020 年 USENIX 年度技术会议 (USENIX ATC 20), 第 307-321 页, 2020. [40] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki 和 Ricardo Bianchini. Splitwise: 使用相分裂的高效生成 llm 推理。2024 年 ACM/IEEE 第 51 届计算机架构国际研讨会 (ISCA), 第 118-132 页。IEEE, 2024. [41] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal 和 Jeff Dean. 有效扩展变压器推理。机器学习和系统论文集, 5, 2023 年。[42] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever 等人。语言模型是无监督的多任务学习者。OpenAI 博客, 1(8):9, 2019. [43] Andrei Radulescu 和 Arjan JC Van Gemund. 异构系统中快速有效的任务调度。第九届异构计算研讨会论文集 (HICW 2000) (目录号 PR00556), 第 229-238 页。IEEE, 2000. [44] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin 等。Code llama: 开放代码基础模型。arXiv 预印本 arXiv:2308.12950, 2023. [45] Max Ryabinin, Tim Dettmers, Michael Diskin 和 Alexander Borzunov. 集群并行性: 训练大型模型的通信效率令人惊讶。国际机器学习会议, 第 29416-29440 页。PMLR, 2023. [46] 马克斯·里亚宾宁和安东·古谢夫。使用分散的专家组合对大型神经网络进行众包训练。神经信息处理系统的进展, 33: 3659–3672, 2020. [47] 诺姆·沙泽尔。快速变压器解码: 您只需要一个写入头。arXiv 预印本 arXiv:1911.02150, 2019. [48] Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhu, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy 和 Ravi Sundaram. Nexus: 一个 GPU 集群引擎, 用于加速基于 DNN 的视频分析。第 27 届 ACM 操作系统原理研讨会论文集, 第 322-337 页, 2019 年。[49] Mohammad Shoyebi, Mostafa Patwary, Rahul Puri, Patrick LeGresley, Jared Casper 和 Bryan Catanzaro. Megatron-lm: 使用模型并行性训练数十亿参数语言模型。arXiv 预印本 arXiv:1909.08053, 2019. [50] Foteini Strati, Paul Elvinger, Tolga Kerimglu 和 Ana Klimovic. 云 GPU 短缺的机器学习训练: 跨区域是答案吗? 第四届机器学习和系统研讨会论文集, 第 107-116 页, 2024 年。[51] Seyed Mohammadhossein Tabatabaee, Jean-Yves Le Boudec 和 Marc Boyer. 交错加权循环: 网络演算分析。IEICE Transactions on Communications, 104(12):1479–1493, 2021. [52] ZeroMQ 作者。ZeroMQ 开源通用消息库, 2024 年。[53] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali 和 Guoqing Harry Xu. Bamboo: 使抢占式实例具有弹性, 可以负担得起大型 DNN 的训练。第 20 届 USENIX 网络系统设计和实现研讨会 (NSDI 23), 第 497-513 页, 马萨诸塞州波士顿, 2023 年 4 月。USENIX 协会。

[54] Haluk Topcuoglu, Salim Hariri 和 Min-You Wu. 用于异构计算的高性能且低复杂度的任务调度。并行和分布式系统上的 IEEE 事务, 13(3):260–274, 2002. [55] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar 等。Llama: 开放高效的基础语言模型。arXiv 预印本 arXiv:2302.13971, 2023. [56] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale 等。Llama 2: 开放基础和微调的聊天模型。arXiv 预印本 arXiv:2307.09288, 2023. [57] 王珏、陆玉成、袁斌航、陈贝迪、梁波西、克里斯托弗·德萨、克里斯托弗·雷和张策。CocktailsGd: 在 500mbps 网络上微调基础模型。国际机器学习会议, 第 36058-36076 页。PMLR, 2023. [58] 吴冰洋, 张自立, 白志豪, 刘轩哲, 金鑫。容器云中透明的 {GPU} 共享用于深度学习工作负载。第 20 届 USENIX 网络系统设计和实现研讨会 (NSDI 23), 第 69-85 页, 2023 年。[59] x.ai。宣布格鲁克。[60] 杨宗衡, 吴章豪, 迈克尔·罗, 蒋伟林, 罗米尔·巴德瓦吉, Woosuk Kwon, 庄思源, 弗兰克·斯飞·栾, 高塔姆·米塔尔, 斯科特·申克, 等。{SkyPilot}: 天空的云间代理

计算。第 20 届 USENIX 网络系统设计与实现研讨会 (NSDI 23), 第 437–455 页, 2023 年。[61] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim 和 Byung-Gon Chun。Orca: 用于{基于 Transformer} 生成模型的分布式服务系统。第 16 届 USENIX 操作系统设计与实现研讨会 (OSDI 22), 第 521-538 页, 2022 年。[62] Binhang Yuan, Yongjun He, Jared Davis, Tianyi 张, Tri Dao, Beidi Chen, Percy S Liang, Christopher Re 和 Ce Zhu。异构环境中基础模型的分散训练。神经信息处理系统进展, 35: 25464–25477, 2022. [63] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan。{MARK}: 利用云服务实现{经济有效的}、{SLO 感知}机器学习推理服务。2019年USENIX年度技术会议 (USENIX ATC 19), 第1049-1062页, 2019. [64] Shiwei Zhang, Lansong Diao, Chuan Wu, Zongyan Cao, Siyu Wang 和 Wei Lin。Hap: 通过自动程序合成对异构 GPU 集群进行 Spmd DNN 训练。第十九届欧洲计算机系统会议论文集, 第 524-541 页, 2024 年。[65] Yinmin Zhu, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin 和 Hao Zhang。{DistServe}: 分解预填充和解码, 以实现良好的优化大型语言模型服务。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24), 第 193-210 页, 2024 年。